

🔗 HIRING MANAGER INTERVIEW PREPARATION

Role: Technical Program Manager - Field Engineering

Job ID: 10409260

Candidate: Juan Murillo

Interview Phase: Phase 1 - Hiring Manager Round

Date Prepared: May 19, 2026

EXECUTIVE SUMMARY FOR HIRING MANAGER

Your Positioning Statement

"I'm a Technical Program Manager with 8+ years orchestrating complex, multi-disciplinary projects. I've led teams across infrastructure, software, AWS, and eCommerce domains. Field Engineering is fundamentally a complex program management challenge—coordinating architects, engineers, and contractors across multiple building systems. I excel at making technical complexity manageable and driving teams toward delivery. That's what Field Engineering needs."

Why You're a Fit for Field Engineering

- ✓ **Complex Program Management:** 5 concurrent projects, 18 engineers, multiple domains
- ✓ **Multi-Discipline Coordination:** Worked with developers, architects, QA, infrastructure, ops
- ✓ **Technical Understanding:** AWS architecture, systems design, infrastructure knowledge
- ✓ **Delivery Excellence:** 100% on-time delivery, quality focus, stakeholder management
- ✓ **Large Team Leadership:** Managed up to 21 engineers across multiple teams
- ✓ **Process Thinking:** Standardization, efficiency improvements, risk management

Hiring Manager's Likely Focus

The Hiring Manager will assess: 1. **Can you orchestrate complex technical projects?** (Core role) 2. **Can you coordinate multiple technical disciplines?** (Architecture + MEP + Infrastructure) 3. **Do you understand infrastructure and systems?** (Baseline knowledge) 4. **Will you drive delivery despite technical complexity?** (Execution capability)

CRITICAL GAPS & YOUR RESPONSE STRATEGY

The Gap: MEP Infrastructure Knowledge

Field Engineering involves MEP (Mechanical, Electrical, Plumbing) infrastructure, infrastructure design, building systems. You have AWS and IT infrastructure knowledge, but not MEP expertise.

Your Strategy: - **Acknowledge:** "I haven't worked directly with MEP systems, but I understand infrastructure complexity. I've managed AWS infrastructure projects involving architects, sysadmins, and complex technical decisions." - **Leverage:** "The principle is the same —understanding what each discipline needs, coordinating them effectively, and driving integration. I've done this across software, infrastructure, and multi-domain projects." - **Commitment:** "MEP knowledge is achievable. I'll work with your architects and engineers to understand building system constraints and interdependencies. I learn quickly through immersion and asking good questions."

Amazon Leadership Principles Most Relevant to Field Engineering

1. **Ownership** - Responsible for end-to-end delivery
 2. **Bias for Action** - Moving forward despite technical uncertainty
 3. **Dive Deep** - Understanding technical complexity before deciding
 4. **Think Big** - Seeing field projects as strategic infrastructure investments
 5. **Simplify** - Making complex multi-discipline projects manageable
 6. **Earn Trust** - Credibility with architects and engineers
-

HIRING MANAGER INTERVIEW: QUESTION & ANSWER PREP

STAR STORY SET 1: COMPLEX PROGRAM MANAGEMENT WITH MULTIPLE TECHNICAL DOMAINS

Question 1: "Tell me about a time you managed a complex project that involved multiple technical disciplines or domains. How did you keep all the pieces coordinated?"

SITUATION:

At Contractor (2020-2022), I led AWS migration projects for three business units that required coordinating multiple technical disciplines:

The Complexity: - **Development teams:** Building features, managing code, testing - **Infrastructure teams:** Provisioning AWS resources, managing networks, capacity planning - **Security team:** Reviewing architecture, approving changes, compliance - **Operations team:** Supporting production, monitoring, incident management - Challenge: Different languages, different priorities, different expertise

TASK:

1. Deliver AWS migration successfully 2. Coordinate 4+ distinct technical disciplines 3. Prevent any discipline from becoming a bottleneck 4. Maintain quality and safety across all domains

ACTION:

I created a multi-discipline coordination framework:

1. **Understanding Phase (2 weeks):**
 - Met with lead in each discipline (dev, infra, security, ops)
 - Understood their constraints, requirements, and risks
 - Identified what could be standard vs. customized
 - Built relationships (critical for cross-discipline work)
2. **Architecture Design (4 weeks):**
 - Brought all disciplines into design conversations
 - Each discipline designed their part
 - Held integration sessions to ensure compatibility
 - Documented assumptions and handoffs
 - Got explicit sign-off from each discipline
3. **Execution Coordination (ongoing):**
 - Weekly all-hands (all 4 disciplines together)
 - Shared vision and progress
 - Each discipline reported status and blockers
 - Identified cross-discipline risks early
 - Made decisions together (not sequentially)
4. **Interface Definition:**
 - Development → Infrastructure: API contracts, resource requirements
 - Infrastructure → Security: Architecture review, approval gates
 - Security → Operations: Monitoring requirements, incident procedures
 - Clear handoff points, no guessing
5. **Problem Resolution:**
 - When disciplines disagreed: technical deep-dive meeting
 - Example: Dev wanted 100ms latency, Infra said network max was 150ms
 - Solution: Optimized caching logic (dev change) + network optimization (infra change)
 - Both disciplines got to yes
 - Always: data-driven decision making
6. **Measurement:**
 - All disciplines met delivery timeline: ✓
 - Quality metrics: maintained across all domains
 - Post-deployment: smooth handoff to ops (no surprises)
 - Cross-discipline satisfaction: positive feedback

RESULT:

- ✓ AWS migration delivered successfully - ✓ All 4 disciplines coordinated effectively - ✓ No cross-discipline delays - ✓ Quality maintained across all domains - ✓ Became model for how to coordinate multiple technical domains

Leadership Principles Demonstrated: - **Dive Deep:** Understood each discipline's unique needs - **Ownership:** Owned overall coordination, not just own discipline - **Think Big:** Saw project as strategic, not just technical execution - **Simplify:** Made complex multi-domain coordination manageable

Why This Works for Field Engineering:

Field Engineering coordinates multiple technical disciplines (architects, MEP engineers, infrastructure, construction). This shows you can: - Understand different technical domains without being expert in all - Coordinate across disciplines effectively - Manage interfaces between technical domains - Solve problems that cross discipline boundaries - Maintain quality across multiple technical areas

STAR STORY SET 2: TECHNICAL COMPLEXITY & SYSTEMS THINKING

Question 2: "Tell me about a time you had to understand and explain complex technical architecture to stakeholders. How did you break down the complexity?"

SITUATION:

At Global Hitss (2018-2019), I had to design and execute AWS infrastructure migration for a product that required complex technical architecture decisions. The challenge: different stakeholders needed different explanations.

TASK:

1. Design AWS architecture that worked technically 2. Explain it to technical teams (architects, engineers) 3. Explain it to business stakeholders (executives, product) 4. Get buy-in from all parties

ACTION:

I created a multi-layer explanation approach:

1. **Technical Architecture Layer (for engineers):**
 - Drew actual AWS architecture
 - Showed services, connections, data flows
 - Explained trade-offs: multi-AZ vs. single AZ, RDS vs. DynamoDB, etc.
 - Discussed failure scenarios and recovery
 - Detailed implementation approach
2. **Business Value Layer (for executives):**
 - Explained cost model: current vs. AWS
 - Showed timeline and resource requirements
 - Discussed risk mitigation
 - ROI analysis
 - Competitive advantages
3. **Systems Thinking Layer (for technical leads):**
 - How does this architecture scale as we grow?
 - How do new services integrate?
 - What are the bottlenecks?
 - How do we monitor and optimize?
 - What does operations look like?

4. **Simplification Approach:**

- Used diagrams (visual > text)
- Analogies to things people understood
- Example: “VPC is like a network security boundary, subnets are like departments”
- Avoided jargon where possible
- Explained WHY each choice, not just WHAT

5. **Getting Buy-In:**

- Presented architecture, asked questions
- Addressed concerns immediately
- Adjusted based on feedback
- Showed how architecture addressed their specific needs
- Got explicit sign-off before proceeding

6. **Measurement:**

- Technical team: confident in architecture
- Business leadership: approved budget and timeline
- Operations team: could implement and support
- All stakeholders understood trade-offs

RESULT:

- ✓ Complex architecture made understandable - ✓ Stakeholders from all areas felt heard - ✓ Buy-in from technical and business teams - ✓ Implementation smooth (architecture was clear) - ✓ Became pattern for explaining technical complexity

Leadership Principles Demonstrated: - **Dive Deep:** Understood technical details deeply - **Simplify:** Made complexity accessible to all audiences - **Earn Trust:** Built credibility through clear communication - **Think Big:** Showed how architecture fit into bigger strategy

Why This Works for Field Engineering:

Field Engineering requires explaining complex architecture to diverse teams. This shows you can: - Understand technical complexity deeply - Explain it to different audiences - Make trade-off decisions clear - Build alignment across technical and business stakeholders - Use systems thinking to guide decisions

STAR STORY SET 3: MANAGING RISK IN COMPLEX TECHNICAL PROJECTS

Question 3: “Tell me about a time you had to manage significant technical risk in a complex project. How did you identify and mitigate it?”

SITUATION:

At Contractor (2020), I was managing payment gateway migration—high-risk because any failure meant financial loss, customer impact, compliance issues.

Context: - Financial transactions at stake (millions of dollars daily) - PCI compliance requirements - Zero downtime requirement - Vendor change required (not just migration) - Multiple systems interconnected (any could break others)

TASK:

1. Migrate payment gateway with zero tolerance for failure 2. Manage multiple interconnected risks 3. Maintain compliance and security 4.

Keep business informed and confident

ACTION:

I created a risk management framework:

1. **Risk Identification (2 weeks):**
 - Brought together dev, infra, security, ops teams
 - Brainstormed what could go wrong at each step
 - Identified risks: migration failure, data loss, integration breaks, performance issues, security gaps
 - Rated by probability and impact
2. **Risk Prioritization:**
 - Highest risk: Migration rollback (if something broke, could we go back?)
 - High risk: Integration failure (payment gateway to other systems)
 - High risk: Performance (transactions taking too long)
 - Medium risk: Data consistency (payments recorded accurately)
3. **Mitigation Strategy:**
 - **For rollback risk:** Created automated rollback procedure, tested weekly
 - **For integration:** Built extensive test suite, did integration testing weekly
 - **For performance:** Load tested migration approach, optimized before production
 - **For data consistency:** Built reconciliation process, continuous monitoring
4. **Execution Approach:**
 - Parallel running (old and new system simultaneously)
 - Gradual traffic shift (start at 5%, increase as confidence grew)
 - 24/7 monitoring and rapid response team (incident command ready)
 - Clear escalation path (any issue → immediate escalation)
5. **Risk Communication:**
 - Weekly risk review with all teams
 - Monthly risk status to leadership
 - Clear risk dashboard (what are we monitoring?)
 - Transparent about remaining risks
6. **Measurement:**
 - Risk scorecard: Identified X risks, mitigated 95%+, remaining 5% monitored
 - Migration: Zero failures, zero data loss
 - Compliance: Passed audit post-migration
 - Business: Confident in process and result

RESULT:

- ✓ Zero migration failures - ✓ Zero data loss or corruption - ✓ Maintained compliance and security - ✓ Business confidence in approach - ✓ Became model for high-risk migration projects

Leadership Principles Demonstrated: - **Ownership:** Owned risk management, not just technical execution - **Dive Deep:** Understood root causes of risks - **Bias for Action:** Moved forward despite risks (through mitigation) - **Earn Trust:** Kept all stakeholders informed and confident

Why This Works for Field Engineering:

Field projects involve significant risks (construction, infrastructure, safety). This shows you can: - Identify technical risks systematically -

Design mitigation strategies that work - Communicate risk clearly to stakeholders - Maintain confidence during high-risk execution - Learn and improve risk management over time

STAR STORY SET 4: PROCESS IMPROVEMENT IN COMPLEX TECHNICAL ENVIRONMENT

Question 4: “Tell me about a time you improved a technical process or workflow that was complex. How did you get teams to adopt the improvement?”

SITUATION:

At Tecnosoftware (2020), the deployment process was complicated and error-prone: - Manual steps (lots of places to make mistakes) - No clear path from development to production - Different teams had different practices - Deployments took 2+ days (slow) - Rollbacks were painful (usually manual)

TASK:

1. Improve deployment process 2. Reduce deployment time 3. Improve reliability (fewer errors) 4. Get teams to actually use it

ACTION:

I redesigned the deployment process with the teams:

1. **Current State Analysis (1 week):**
 - Documented actual deployment process (not assumed)
 - Found bottlenecks: manual testing (1 day), approval process (1 day), actual deployment (4 hours)
 - Identified pain points: error-prone manual steps, lack of visibility, slow rollbacks
2. **Future State Design (with teams):**
 - Automated testing (remove manual testing)
 - Clear approval gates (remove gatekeeping)
 - Infrastructure as code (enable rollback)
 - Monitoring (know if deployment succeeded)
3. **Implementation (phased):**
 - Phase 1: Automated testing (dev team led this)
 - Phase 2: Approval process redesign (product team + security)
 - Phase 3: Infrastructure as code (infra team)
 - Phase 4: Deployment automation (infra team)
4. **Adoption Strategy:**
 - Trained all teams (2 hours training per team)
 - First deployment with new process together (supported, not solo)
 - Identified and celebrated early wins
 - Collected feedback and adjusted
 - After 3 deployments, teams felt confident
5. **Measurement:**
 - Deployment time: 2 days → 4 hours (95% improvement)
 - Errors/rollbacks: 2-3 per quarter → 1 per quarter
 - Visibility: teams knew status in real-time
 - Team satisfaction: process felt much better

RESULT:

- ✓ 95% reduction in deployment time - ✓ Dramatic improvement in reliability - ✓ Teams adopted and preferred new process - ✓ Became standard for all future deployments

Leadership Principles Demonstrated: - **Ownership:** Owned process improvement, not just implementation - **Simplify:** Made complex process much simpler - **Bias for Action:** Implemented improvements despite complexity - **Earn Trust:** Built trust by improving actual pain points

Why This Works for Field Engineering:

Field Engineering manages complex processes (design, construction, operations). This shows you can: - Understand complex workflows - Improve them without breaking what's working - Get teams to adopt improvements - Measure success appropriately - Iterate based on feedback

INTERVIEW DELIVERY STRATEGY

The Hiring Manager's Perspective

The Hiring Manager will be listening for:

1. Can you manage technical complexity? → Story 1 & 3 demonstrate this. Emphasize: - How you break down complexity - How you coordinate across disciplines - How you make good technical decisions

2. Do you understand technical architecture? → Story 2 directly demonstrates this. Emphasize: - Your systems thinking - Your ability to explain trade-offs - Your communication to different audiences

3. Can you manage risk in complex projects? → Story 3 demonstrates this. Emphasize: - How you identify risks systematically - How you mitigate without over-engineering - How you keep stakeholders confident

4. Can you improve processes in technical environments? → Story 4 demonstrates this. Emphasize: - How you work WITH teams (not to them) - How you improve outcomes significantly - How you measure success

FREQUENTLY ASKED FOLLOW-UP QUESTIONS & ANSWERS

Follow-up Q: "How would you learn about MEP systems?"

Answer: "Three approaches: (1) work closely with your MEP engineers—sit in design reviews, ask questions about trade-offs, (2) study documentation and learn the basics, (3) visit actual field locations to understand physical reality. I'll be very hands-on learning."

In 4-6 weeks, I'll have working knowledge. In 3 months, I'll understand how MEP decisions affect field engineering timeline and cost."

Follow-up Q: "What would you do in your first 90 days?"

Answer: "First 30 days: listen. Understand the current projects, talk to the team, understand architecture and MEP constraints. Second 30 days: dive deep on one project—understand design, construction timeline, risks. Third 30 days: proactive contributions—identify improvements, propose process changes, start driving results. By 90 days, I'll have context to be fully effective."

Follow-up Q: "How do you handle architects who resist your project management approach?"

Answer: "I earn their trust first by understanding their needs and constraints. Good architects want good project management—it gives them the space to do good work. My approach is to remove blockers, coordinate dependencies, and keep timelines realistic. Architects will support that. I don't impose—I collaborate."

PRACTICE CHECKLIST

Before your interview:

- ☐ **Record yourself** answering all 4 stories
 - ☐ **Time yourself** - 90-120 seconds per story
 - ☐ **Listen for clarity**, specificity, confidence
 - ☐ **Study MEP basics** - understand foundational concepts
 - ☐ **Practice with someone** who can ask follow-ups
 - ☐ **Prepare questions** to ask Hiring Manager
 - ☐ **Iterate** - refine weak stories
 - ☐ **Practice 5-10 times** before interview
-

KEY MESSAGES TO REINFORCE

Your Differentiator

"I excel at orchestrating complex technical projects involving multiple disciplines. Field Engineering is fundamentally that challenge—coordinating architects, engineers, and teams toward delivery. I've done this across software, infrastructure, and AWS domains."

Your Strength Area

"I'm strongest at making technical complexity manageable through clear communication, systematic coordination, and focus on delivery."

Your Approach

“I believe in working WITH technical teams, not to them. I listen, I learn, I coordinate, I remove blockers. That builds the trust I need to drive results.”

Your Growth Edge

“MEP knowledge is a gap I’ll close through immersion. My track record shows I learn new domains quickly and contribute strategically within weeks.”

FINAL PREPARATION STEPS

Week Before Interview:

1. Record 4 STAR stories (3-4 times)
2. Study MEP fundamentals
3. Practice with follow-up questions
4. Prepare your questions

Day Before:

1. Review key numbers
2. Practice once
3. Sleep well

Day Of:

1. Arrive early
 2. Greet warmly
 3. Be authentic
 4. Show enthusiasm
-

SUCCESS INDICATORS

After your interview, the Hiring Manager should think:

- ✓ “This person can orchestrate complex technical projects”
- ✓ “They understand technical systems and trade-offs”
- ✓ “They can coordinate multiple technical disciplines”
- ✓ “They’ll own delivery and keep teams aligned”
- ✓ “They learn quickly and will pick up MEP knowledge”
- ✓ “They have the leadership capability we need”

Your goal is to make ALL of these obvious.

Preparation Status: Ready for Interview

Confidence Level Target: 8/10+

Next Step: Record yourself, get feedback, iterate

You’ve got this. Show them your technical orchestration superpower.



END OF HIRING MANAGER INTERVIEW PREP